Sheet 6: Training a RNN to learn sinusoidal oscillations

In [11]:

```python
import torch as tc
import torch.nn as nn
import torch.optim as optim
import numpy as np
from matplotlib import pyplot as plt
```

```
/Users/samrouppe/opt/anaconda3/envs/3dcv-students/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not fo
und. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

The order of the tasks might be a bit shuffled, we marked them where there was text to write.

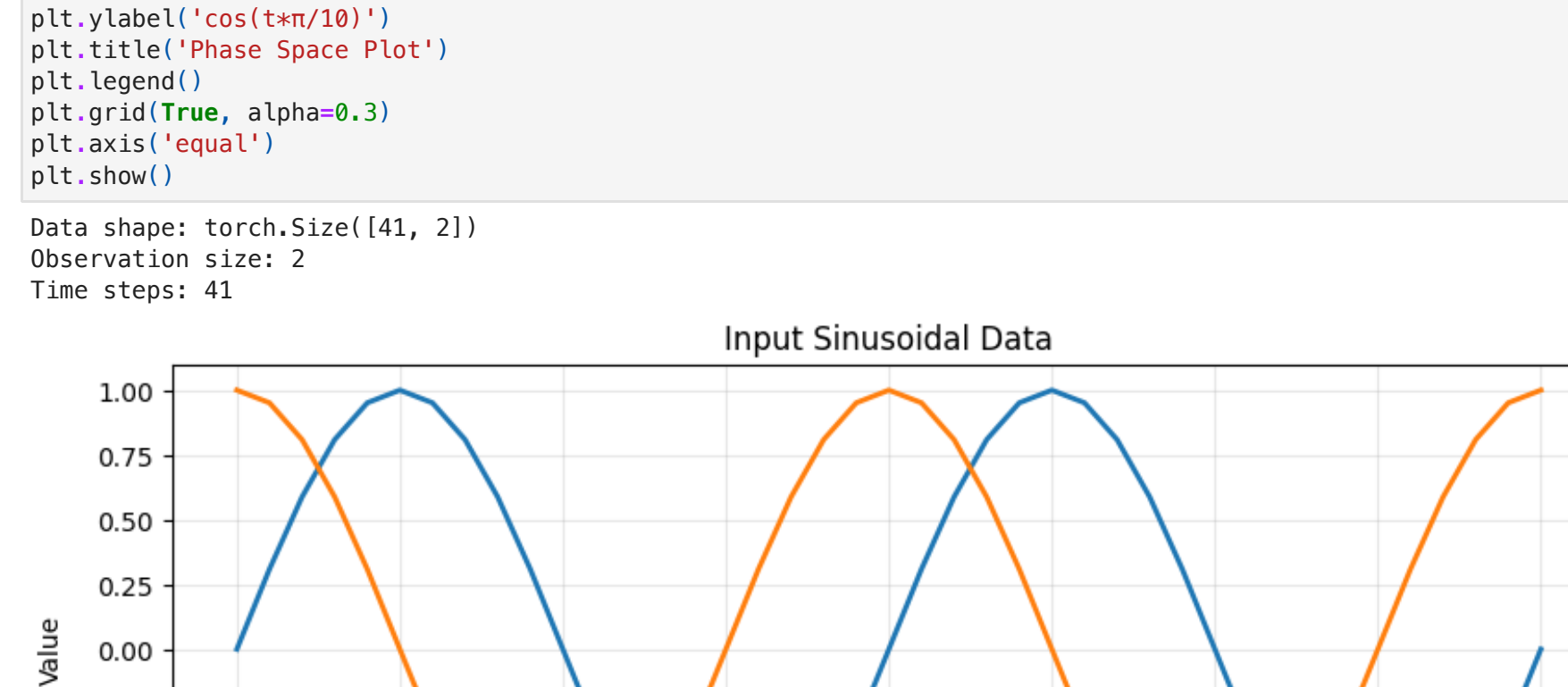
In [ ]:

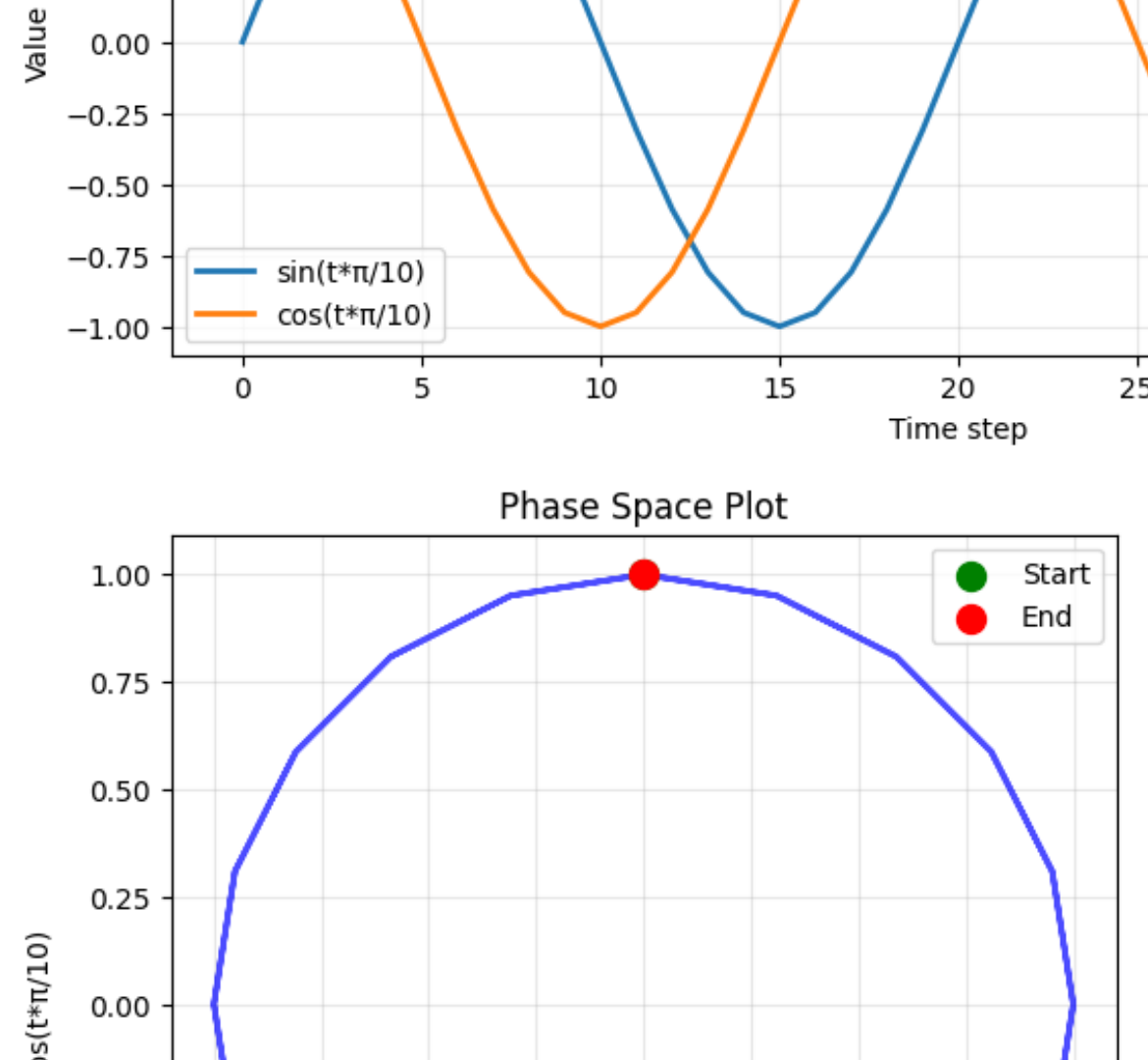
```python
# TASK 2: Set the hidden size (dimension of z_t)
# Try different values: start with 2, then 4, 8, 16...
# Find the minimum number that works well
hidden_size = 4 # TODO: Replace with your choice

# Training parameters
epochs = 2000
learning_rate = 1e-4 # TODO: Try different values

# TASK 4: Mini-batching parameters (uncomment and modify when needed)
seq_length = 15     # Length of sampeld subsequences for training
batch_size = 5     # Number of subsequences per batch
```

In [13]:

```python
#%% DATA LOADING AND VISUALIZATION

# Load the sinusoidal data: x_t = [sin(t*π/10), cos(t*π/10)] for t=0,...,40
data = tc.load('sinus.pt')
observation_size = data.shape[1]  # Should be 2 (sin and cos components)

print(f"Data shape: {data.shape}")
print(f"Observation size: {observation_size}")
print(f"Time steps: {data.shape[0]}")

# Plot the input data
plt.figure(figsize=(10, 4))
plt.plot(data[:, 0], label='sin(t*π/10)', linewidth=2)
plt.plot(data[:, 1], label='cos(t*π/10)', linewidth=2)
plt.xlabel('Time step')
plt.ylabel('Value')
plt.title('Input Sinusoidal Data')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Plot phase space (sin vs cos)
plt.figure(figsize=(6, 6))
plt.plot(data[:, 0], data[:, 1], 'b-', linewidth=2, alpha=0.7)
plt.scatter(data[0, 0], data[0, 1], c='green', s=100, label='Start', zorder=5)
plt.scatter(data[-1, 0], data[-1, 1], c='red', s=100, label='End', zorder=5)
plt.xlabel('sin(t*π/10)')
plt.ylabel('cos(t*π/10)')
plt.title('Phase Space Plot')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()
```

```
Data shape: torch.Size([41, 2])
Observation size: 2
Time steps: 41
```


Input Sinusoidal Data


Phase Space Plot

In [14]:

```python
#%% MODEL DEFINITION

class LatentRNN(nn.Module):
    """
    Recurrent Neural Network for learning dynamical systems

    Architecture:
    z_t = tanh(C * x_{t-1} + W * z_{t-1} + h)  # Hidden state update
    \hat{x}_t = B * z_t + c                    # Output generation
    """

    def __init__(self, obs_dim, latent_dim):
        super(LatentRNN, self).__init__()

        self.obs_dim = obs_dim        # Dimension of observations (2 for sin/cos)
        self.latent_dim = latent_dim  # Dimension of hidden state z_t
        self.activation = nn.Tanh()

        # TASK 2: Implement the RNN layers here

        # Option 1: Use nn.RNN (easier)
        #or
        # Option 2: Implement manually with nn.Linear layers (more educational)
        # You need to define the following components:
        # 1. Input-to-hidden transformation: U matrix and bias b
        # 2. Hidden-to-hidden transformation: V matrix
        # 3. Hidden-to-output transformation: W matrix and bias c
        # Hint: You can use nn.Linear layers or define nn.Parameter matrices

        # TODO: Implement the network architecture
        ####### own implementation
        # Input-to-hidden (U * x + b)
        self.input_to_hidden = nn.Linear(obs_dim, latent_dim, bias=True)
        # Hidden-to-hidden (W * z)
        self.hidden_to_hidden = nn.Linear(latent_dim, latent_dim, bias=False)

        #self.rnn = nn.RNN(obs_dim, latent_dim, 1,nonlinearity='tanh') # using nn.RNN

        # Hidden-to-output (B * z + c)
        self.hidden_to_output = nn.Linear(latent_dim, obs_dim, bias=True)


    def forward(self, time_series, h0):
        """
        Forward pass through the RNN

        Args:
            time_series: Input sequence of shape (seq_len, batch_size, obs_dim)
            h0: Initial hidden state of shape (1, batch_size, latent_dim)

        Returns:
            obs_output: Predicted observations of shape (seq_len, batch_size, obs_dim)
            h: Final hidden state of shape (1, batch_size, latent_dim)
        """

        # TASK 2: Implement the forward pass
        # The output should be predicted observations and the final hidden state

        # Hint: For each time step, update hidden state and generate output

        # TODO: Implement the forward loop

        ####### own implementation
        seq_len, batch_size, _ = time_series.shape
        h = h0
        outputs = []

        for t in range(seq_len):
            x_t = time_series[t]  # shape: (batch_size, obs_dim)
            h = self.activation(self.input_to_hidden(x_t) + self.hidden_to_hidden(h))  # z_t
            x_hat = self.hidden_to_output(h)  # \hat{x}_t
            outputs.append(x_hat)

        obs_output = tc.stack(outputs, dim=0)  # (seq_len, batch_size, obs_dim)

        #rnn_out, h = self.rnn(time_series, h0)  # using nn.RNN
        #obs_output = self.hidden_to_output(rnn_out)  # apply output projection at each timestep
        return obs_output, h


# Initialize the model
model = LatentRNN(observation_size, hidden_size)
# Print model information
print(f"\nModel Architecture:")
print(f"- Observation dimension: {observation_size}")
print(f"- Hidden dimension: {hidden_size}")
print(f"- Total parameters: {sum(p.numel() for p in model.parameters())}")
```

```
Model Architecture:
- Observation dimension: 2
- Hidden dimension: 4
- Total parameters: 38
```

In [25]:

```python
def train(model,learning_rate,hidden_size,epochs=epochs, moment=0, optimizer_function='SGD', print_loss=True, batch_size=1, b
    """
    Training function with configurable optimizers and mini-batching

    Students need to implement the missing parts marked with TODO comments.
    """

    # TODO: Implement optimizer selection
    # Create the appropriate optimizer based on optimizer_function parameter
    # Available options: 'SGD' (with momentum support) and 'ADAM'
    # Hint: Use optim.SGD() and optim.Adam() from torch.optim
    parameters = list(model.parameters())

    if optimizer_function == 'SGD':
        # TODO: Initialize SGD optimizer with learning_rate and momentum
        optimizer = optim.SGD(parameters, lr=learning_rate, momentum=moment)
    elif optimizer_function == 'ADAM':
        # TODO: Initialize Adam optimizer with learning_rate
        optimizer = optim.Adam(parameters, lr=learning_rate)
    else:
        raise ValueError(f"Unknown optimizer: {optimizer_function}")

    # TODO: Define the loss function
    # Use Mean Squared Error (MSE) loss for this regression task
    loss_function = nn.MSELoss()

    losses = []
    if print_loss:
        print(f"\nStarting training for {epochs} epochs...")
        print(f"Optimizer: {optimizer_function}, LR: {learning_rate}, Batch size: {batch_size}, Sequence length: {batch_seque

    for epoch in range(epochs):
        # TODO: Initialize hidden state for the batch
        # Create a tensor of shape (1, batch_size, hidden_size) with random values
        if minibatch:
            h0 = tc.rand(batch_size, hidden_size)
            h = h0
        else:
            h0 = tc.rand(1, 1, hidden_size)
            h = h0

        # Prepare full sequences (input and target)
        x = data[:-1]  # Input: all timesteps except the last
        y = data[1:]   # Target: all timesteps except the first

        # TODO: Create batch tensors for mini-batching
        # Initialize tensors to hold batch data of shape (batch_sequence_length, batch_size, observation_dim):
        X = tc.zeros(batch_sequence_length, batch_size, observation_size)
        Y = tc.zeros(batch_sequence_length, batch_size, observation_size)

        # TODO: Implement mini-batching
        # For each element in the batch, sample a random subsequence from the data
        # The subsequence should start at a random index and have length batch_sequence_length
        # Make sure the random index doesn't exceed the data bounds
        if minibatch:
            for j in range(batch_size):
                # TODO: Sample a random starting index for the subsequence
                # Ensure: 0 <= ind <= len(x) - batch_sequence_length
                ind = tc.randint(0, len(x) - batch_sequence_length + 1, (1,)).item()
                end_ind = ind+batch_sequence_length
                # TODO: Extract subsequence and assign to batch tensors
                X[:,j,:] = x[ind:end_ind]
                Y[:,j,:] = y[ind:end_ind]

        # TODO: Forward pass
        # 1. Zero the gradients from previous iteration
        # 2. Run the model forward pass with input X and initial hidden state h0
        # 3. Calculate the loss between model output and target Y
        optimizer.zero_grad()

        if minibatch:
            obs_output, h = model(X,h)
            epoch_loss = loss_function(obs_output,Y)
        else:
            obs_output, h = model(x.unsqueeze(1),h)
            epoch_loss = loss_function(obs_output.squeeze(1).squeeze(1),y)



        # TODO: Backward pass and optimization step
        # 1. Compute gradients via backpropagation
        # 2. Update model parameters
        epoch_loss.backward()

        optimizer.step()

        # Store loss for plotting
        losses.append(epoch_loss.item())

        # Print progress
        if epoch % 10 == 0 and print_loss:
            print(f"Epoch: {epoch} loss {epoch_loss.item():.6f}")

    return losses
```

In [16]:

```python
import matplotlib.pyplot as plt

def plot_results(arrays, hyperparams_list, nrows=1, ncols=None, log_y=False):
    """
    Plots each array in `arrays` as a subplot using the specified grid layout,
    with titles showing the corresponding hyperparameters.

    Parameters:
    - arrays: list of 1D numpy arrays or torch tensors representing y-values
    - hyperparams_list: list of dicts with keys:
        'learning_rate', 'moment', 'optimizer_function',
        'batch_size', 'batch_sequence_length', 'minibatch'
    - nrows: number of rows in the subplot grid
    - ncols: number of columns in the subplot grid (calculated automatically if None)
    - log_y: bool, if True sets y-axis to logarithmic scale
    """
    n = len(arrays)
    assert len(hyperparams_list) == n, "Each array must have a matching set of hyperparameters."

    if ncols is None:
        ncols = (n + nrows - 1) // nrows  # Auto-fit columns to rows

    fig, axes = plt.subplots(nrows, ncols, figsize=(5 * ncols, 3 * nrows), squeeze=False)

    for idx, (y_vals, hyperparams) in enumerate(zip(arrays, hyperparams_list)):
        row = idx // ncols
        col = idx % ncols
        ax = axes[row][col]

        x_vals = range(len(y_vals))
        ax.plot(x_vals, y_vals)
        #print(hyperparams)
        if log_y:
            ax.set_yscale('log')

        title = f"LR: {hyperparams['learning_rate']}, Opt: {hyperparams['optimizer_function']}, Minibatch: {hyperparams['mini

        if hyperparams['minibatch']:
            title += f"Batch: {hyperparams['batch_size']}, SeqLen: {hyperparams['batch_sequence_length']}, "

        if hyperparams['optimizer_function'] == 'SGD':
            title += f"Momentum: {hyperparams['moment']},"

        ax.set_title(title)
        ax.set_ylabel("Value")
        ax.grid(True)

    # Hide any unused subplots
    for idx in range(n, nrows * ncols):
        fig.delaxes(axes[idx // ncols][idx % ncols])

    for ax in axes[-1]:
        ax.set_xlabel("Step")

    plt.tight_layout()
    plt.show()
```

In [17]:

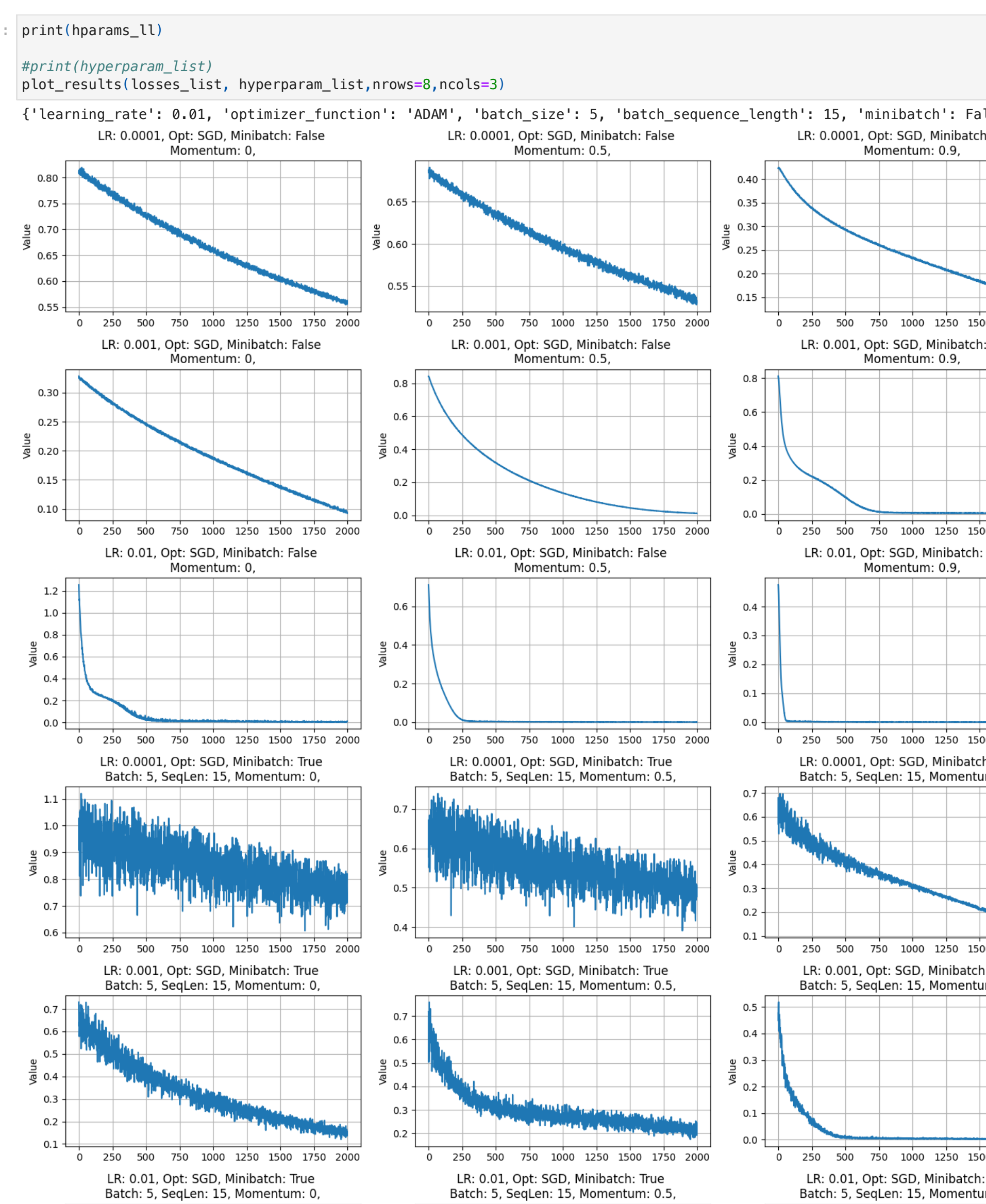
```python
# Train the models
optimisers = ['SGD','ADAM']
minibatch = [False,True]
LR = [1e-4,1e-3,1e-2]
moments=[0,0.5,0.9]

hyperparam_list = []
losses_list = []

min_loss = 10000


for opt in optimisers:
    for b in minibatch:
        for lr in LR:
            if opt == 'SGD':
                for i in moments:
                    model = LatentRNN(observation_size, hidden_size)
                    hparams = {'learning_rate':lr, 'moment':i,'optimizer_function':opt, 'batch_size':5, 'batch_sequence_lengt
                    losses = train(model,lr, hidden_size, optimizer_function=opt,moment=i,
                        print_loss=False, batch_size=5, batch_sequence_length=15,minibatch=b) # TODO: input the right hyperpa
                    losses_list.append(losses)
                    hyperparam_list.append(hparams)
                    if losses[-1] < min_loss:
                        min_loss = losses[-1]
                        print(min_loss)
                        hparams_ll = hparams # least loss hyper params

            else:
                model = LatentRNN(observation_size, hidden_size)
                hparams = {'learning_rate':lr, 'optimizer_function':opt, 'batch_size':5, 'batch_sequence_length':15, 'minibat
                losses = train(model,lr, hidden_size, optimizer_function=opt,
                    print_loss=False, batch_size=5, batch_sequence_length=15,minibatch=b) # TODO: input the right hyperparame
                losses_list.append(losses)
                hyperparam_list.append(hparams)
                if losses[-1] < min_loss:
                    min_loss = losses[-1]
                    print(min_loss)
                    hparams_ll = hparams # least loss hyper params





num_plots = len(minibatch)*len(LR)*len(moments) + len(minibatch)*len(LR)
print(num_plots)
```

```
0.5621148347854614
0.5325733423233032
0.13410326838493347
0.09239083528518677
0.011761186644434929
0.003671844955533743
0.0008285142248496413
0.00020860911172349006
9.682579729997087e-06
24
```

In [18]:

```python
print(hparams_ll)

#print(hyperparam_list)
plot_results(losses_list, hyperparam_list,nrows=8,ncols=3)
```

```
{'learning_rate': 0.01, 'optimizer_function': 'ADAM', 'batch_size': 5, 'batch_sequence_length': 15, 'minibatch': False}
```


LR: 0.0001, Opt: SGD, Minibatch: False Momentum: 0,
LR: 0.0001, Opt: SGD, Minibatch: False Momentum: 0.5,
LR: 0.0001, Opt: SGD, Minibatch: False Momentum: 0.9,
LR: 0.001, Opt: SGD, Minibatch: False Momentum: 0,
LR: 0.001, Opt: SGD, Minibatch: False Momentum: 0.5,
LR: 0.001, Opt: SGD, Minibatch: False Momentum: 0.9,
LR: 0.01, Opt: SGD, Minibatch: False Momentum: 0,
LR: 0.01, Opt: SGD, Minibatch: False Momentum: 0.5,
LR: 0.01, Opt: SGD, Minibatch: False Momentum: 0.9,
LR: 0.0001, Opt: SGD, Minibatch: True Batch: 5, SeqLen: 15, Momentum: 0,
LR: 0.0001, Opt: SGD, Minibatch: True Batch: 5, SeqLen: 15, Momentum: 0.5,
LR: 0.0001, Opt: SGD, Minibatch: True Batch: 5, SeqLen: 15, Momentum: 0.9,
LR: 0.001, Opt: SGD, Minibatch: True Batch: 5, SeqLen: 15, Momentum: 0,
LR: 0.001, Opt: SGD, Minibatch: True Batch: 5, SeqLen: 15, Momentum: 0.5,
LR: 0.001, Opt: SGD, Minibatch: True Batch: 5, SeqLen: 15, Momentum: 0.9,
LR: 0.01, Opt: SGD, Minibatch: True Batch: 5, SeqLen: 15, Momentum: 0,
LR: 0.01, Opt: SGD, Minibatch: True Batch: 5, SeqLen: 15, Momentum: 0.5,
LR: 0.01, Opt: SGD, Minibatch: True Batch: 5, SeqLen: 15, Momentum: 0.9,
LR: 0.0001, Opt: ADAM, Minibatch: False
LR: 0.001, Opt: ADAM, Minibatch: False
LR: 0.01, Opt: ADAM, Minibatch: False
LR: 0.0001, Opt: ADAM, Minibatch: True Batch: 5, SeqLen: 15,
LR: 0.001, Opt: ADAM, Minibatch: True Batch: 5, SeqLen: 15,
LR: 0.01, Opt: ADAM, Minibatch: True Batch: 5, SeqLen: 15,

This is a big figure so we will refer to the plots, using matrix element notation, so (i,j) = (row,column).

here SGD with minibatch = false behaves like regular gradient decent.

3.1

The affect of adjusting learning rate with SGD is seen i row 1-6, column 1. We see that the learning rate manly affects how quicly the loss decreases and in this case the larger learning rate also seems to be more stable going down with less variance per step. For The learning rate with ADAM row 7-8 we see the same, with higher learning the loss goes down faster and with less variance.

3.2

Changing the momentum for SGD with a given learning rate is seen in row 1-6, column 1-3. The increasing momentum seems to increase the absolute value of the slope of the decreasing loss. Which aligns with the expectation that the momentum modulates the learning rate based on the previous gradient values, so with high momentum the modulation becomes stronger. We also see it is not just increasing the absolute value of the slope, like mentioned it modulates the learning rate based on previous gradients (can also decrease abs value of the slope not only increase), and so, for example in figure (2,3) we have an example where the momentum decreases the slope, maybe because of a region with low gradtients, before the slope increases again.

3.3

SGD row 1-6, column 1-3 and ADAM row 7-8 , colum 1-3. Over all the Adam vs SGD runs with the rest of the hyperparameters similar, the ADAM optimiser seems to have a faster converging loss. For example seen when comparing SGD row 5, column 1 and Adam row 8, column 2.

3.4

Figure row 2, column 3 we see that the loss curve flattens out after initial steep decrease, before dropping to to close to zero afterward, this could maybe be a result from a bifurcation, we plot before and after below.

4

overall minibatching seems to improve stability, but for low learning rates it did increase the variance.

Generating 5x data len traj

In [40]:

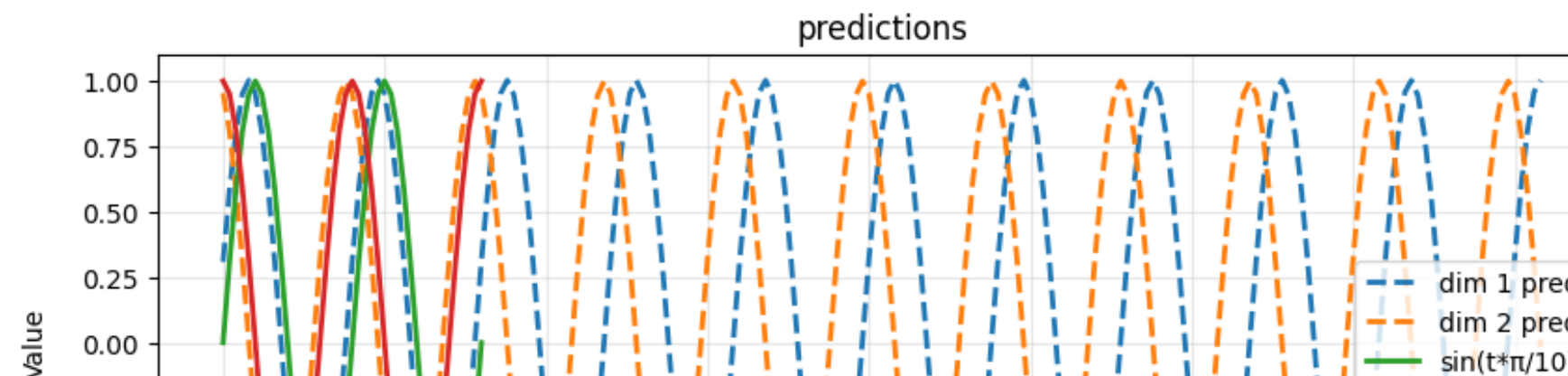
```python
#%% MODEL EVALUATION AND PREDICTION

model = LatentRNN(observation_size, hidden_size)

losses = train(model,hidden_size=hidden_size,**hparams_ll)

# Generate predictions for 5 times the original sequence length
prediction_length = 5 * data.shape[0]

def generate_traj(model,prediction_length):
    with tc.no_grad():
        #h = tc.rand(1, hidden_size).unsqueeze(1)
        h = tc.rand(1, hidden_size)  # shape: (batch_size=1, latent_dim)
        predictions = tc.zeros(prediction_length, 1, observation_size)
        warmup_hidden = data.unsqueeze(1)
        dum, h = model(warmup_hidden, h)

        # Start with first data point
        input_ = data[0:1].unsqueeze(1)
        #input_ = data[-2:-1].unsqueeze(1)
        out, h = model(input_, h)
        predictions[0] = out.squeeze(0)  # remove time dim

        # Autoregressive generation
        for i in range(1, prediction_length):
            input_ = out  # previous output
            out, h = model(input_, h)  # out: (1, 1, obs_dim)
            predictions[i] = out.squeeze(0)  # again, remove time dim
    return predictions.squeeze()
```

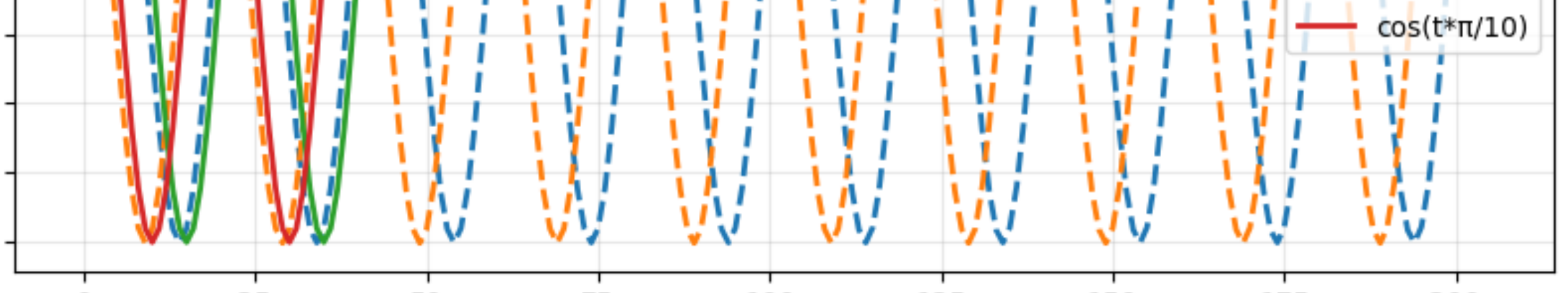
```
Starting training for 2000 epochs...
Optimizer: ADAM, LR: 0.01, Batch size: 5, Sequence length: 15
Epoch: 0 loss 0.559811
Epoch: 10 loss 0.350637
Epoch: 20 loss 0.257761
Epoch: 30 loss 0.208154
Epoch: 40 loss 0.138085
Epoch: 50 loss 0.045347
Epoch: 60 loss 0.022917
Epoch: 70 loss 0.013842
Epoch: 80 loss 0.010335
Epoch: 90 loss 0.008816
Epoch: 100 loss 0.006811
Epoch: 110 loss 0.004992
Epoch: 120 loss 0.005061
Epoch: 130 loss 0.004057
Epoch: 140 loss 0.004429
Epoch: 150 loss 0.003177
Epoch: 160 loss 0.004012
Epoch: 170 loss 0.002741
Epoch: 180 loss 0.002569
Epoch: 190 loss 0.002431
Epoch: 200 loss 0.002215
Epoch: 210 loss 0.001796
Epoch: 220 loss 0.002810
Epoch: 230 loss 0.002193
Epoch: 240 loss 0.001775
Epoch: 250 loss 0.001850
Epoch: 260 loss 0.001782
Epoch: 270 loss 0.001369
Epoch: 280 loss 0.001033
Epoch: 290 loss 0.001217
Epoch: 300 loss 0.001425
Epoch: 310 loss 0.001233
Epoch: 320 loss 0.000942
Epoch: 330 loss 0.001139
Epoch: 340 loss 0.000721
Epoch: 350 loss 0.000581
Epoch: 360 loss 0.000672
Epoch: 370 loss 0.000644
Epoch: 380 loss 0.000535
Epoch: 390 loss 0.000765
Epoch: 400 loss 0.000461
Epoch: 410 loss 0.000305
Epoch: 420 loss 0.000472
Epoch: 430 loss 0.000302
Epoch: 440 loss 0.000383
Epoch: 450 loss 0.000393
Epoch: 460 loss 0.000231
Epoch: 470 loss 0.000160
Epoch: 480 loss 0.000240
Epoch: 490 loss 0.000302
Epoch: 500 loss 0.000186
Epoch: 510 loss 0.000132
Epoch: 520 loss 0.000129
Epoch: 530 loss 0.000111
Epoch: 540 loss 0.000149
Epoch: 550 loss 0.000151
Epoch: 560 loss 0.000274
Epoch: 570 loss 0.000234
Epoch: 580 loss 0.000086
Epoch: 590 loss 0.000156
Epoch: 600 loss 0.000185
Epoch: 610 loss 0.000087
Epoch: 620 loss 0.000082
Epoch: 630 loss 0.000085
Epoch: 640 loss 0.000169
Epoch: 650 loss 0.000099
Epoch: 660 loss 0.000089
Epoch: 670 loss 0.000062
Epoch: 680 loss 0.000113
Epoch: 690 loss 0.000045
Epoch: 700 loss 0.000098
Epoch: 710 loss 0.000058
Epoch: 720 loss 0.000055
Epoch: 730 loss 0.000262
Epoch: 740 loss 0.000045
Epoch: 750 loss 0.000108
Epoch: 760 loss 0.000092
Epoch: 770 loss 0.000034
Epoch: 780 loss 0.000068
Epoch: 790 loss 0.000041
Epoch: 800 loss 0.000151
Epoch: 810 loss 0.000085
Epoch: 820 loss 0.000031
Epoch: 830 loss 0.000050
Epoch: 840 loss 0.000033
Epoch: 850 loss 0.000035
Epoch: 860 loss 0.000066
Epoch: 870 loss 0.000041
Epoch: 880 loss 0.000041
Epoch: 890 loss 0.000105
Epoch: 900 loss 0.000102
Epoch: 910 loss 0.000037
Epoch: 920 loss 0.000133
Epoch: 930 loss 0.000037
Epoch: 940 loss 0.000082
Epoch: 950 loss 0.000038
Epoch: 960 loss 0.000029
Epoch: 970 loss 0.000063
Epoch: 980 loss 0.000134
Epoch: 990 loss 0.000038
Epoch: 1000 loss 0.000078
Epoch: 1010 loss 0.000052
Epoch: 1020 loss 0.000034
Epoch: 1030 loss 0.000091
Epoch: 1040 loss 0.000028
Epoch: 1050 loss 0.000088
Epoch: 1060 loss 0.000028
Epoch: 1070 loss 0.000023
Epoch: 1080 loss 0.000027
Epoch: 1090 loss 0.000029
Epoch: 1100 loss 0.000058
Epoch: 1110 loss 0.000037
Epoch: 1120 loss 0.000054
Epoch: 1130 loss 0.000025
Epoch: 1140 loss 0.000058
Epoch: 1150 loss 0.000029
Epoch: 1160 loss 0.000031
Epoch: 1170 loss 0.000072
Epoch: 1180 loss 0.000052
Epoch: 1190 loss 0.000032
Epoch: 1200 loss 0.000021
Epoch: 1210 loss 0.000041
Epoch: 1220 loss 0.000042
Epoch: 1230 loss 0.000019
Epoch: 1240 loss 0.000022
Epoch: 1250 loss 0.000021
Epoch: 1260 loss 0.000029
Epoch: 1270 loss 0.000054
Epoch: 1280 loss 0.000038
Epoch: 1290 loss 0.000030
Epoch: 1300 loss 0.000039
Epoch: 1310 loss 0.000030
Epoch: 1320 loss 0.000034
Epoch: 1330 loss 0.000049
Epoch: 1340 loss 0.000034
Epoch: 1350 loss 0.000045
Epoch: 1360 loss 0.000022
Epoch: 1370 loss 0.000020
Epoch: 1380 loss 0.000041
Epoch: 1390 loss 0.000038
Epoch: 1400 loss 0.000020
Epoch: 1410 loss 0.000019
Epoch: 1420 loss 0.000026
Epoch: 1430 loss 0.000026
Epoch: 1440 loss 0.000016
Epoch: 1450 loss 0.000017
Epoch: 1460 loss 0.000021
Epoch: 1470 loss 0.000026
Epoch: 1480 loss 0.000021
Epoch: 1490 loss 0.000019
Epoch: 1500 loss 0.000018
Epoch: 1510 loss 0.000022
Epoch: 1520 loss 0.000027
Epoch: 1530 loss 0.000016
Epoch: 1540 loss 0.000016
Epoch: 1550 loss 0.000015
Epoch: 1560 loss 0.000014
Epoch: 1570 loss 0.000017
Epoch: 1580 loss 0.000024
Epoch: 1590 loss 0.000030
Epoch: 1600 loss 0.000025
Epoch: 1610 loss 0.000015
Epoch: 1620 loss 0.000014
Epoch: 1630 loss 0.000015
Epoch: 1640 loss 0.000018
Epoch: 1650 loss 0.000022
Epoch: 1660 loss 0.000014
Epoch: 1670 loss 0.000017
Epoch: 1680 loss 0.000017
Epoch: 1690 loss 0.000014
Epoch: 1700 loss 0.000020
Epoch: 1710 loss 0.000014
Epoch: 1720 loss 0.000015
Epoch: 1730 loss 0.000019
Epoch: 1740 loss 0.000013
Epoch: 1750 loss 0.000016
Epoch: 1760 loss 0.000015
Epoch: 1770 loss 0.000015
Epoch: 1780 loss 0.000014
Epoch: 1790 loss 0.000014
Epoch: 1800 loss 0.000014
Epoch: 1810 loss 0.000012
Epoch: 1820 loss 0.000017
Epoch: 1830 loss 0.000013
Epoch: 1840 loss 0.000015
Epoch: 1850 loss 0.000012
Epoch: 1860 loss 0.000012
Epoch: 1870 loss 0.000013
Epoch: 1880 loss 0.000011
Epoch: 1890 loss 0.000016
Epoch: 1900 loss 0.000015
Epoch: 1910 loss 0.000013
Epoch: 1920 loss 0.000017
Epoch: 1930 loss 0.000011
Epoch: 1940 loss 0.000013
Epoch: 1950 loss 0.000012
Epoch: 1960 loss 0.000011
Epoch: 1970 loss 0.000011
Epoch: 1980 loss 0.000012
Epoch: 1990 loss 0.000011
```

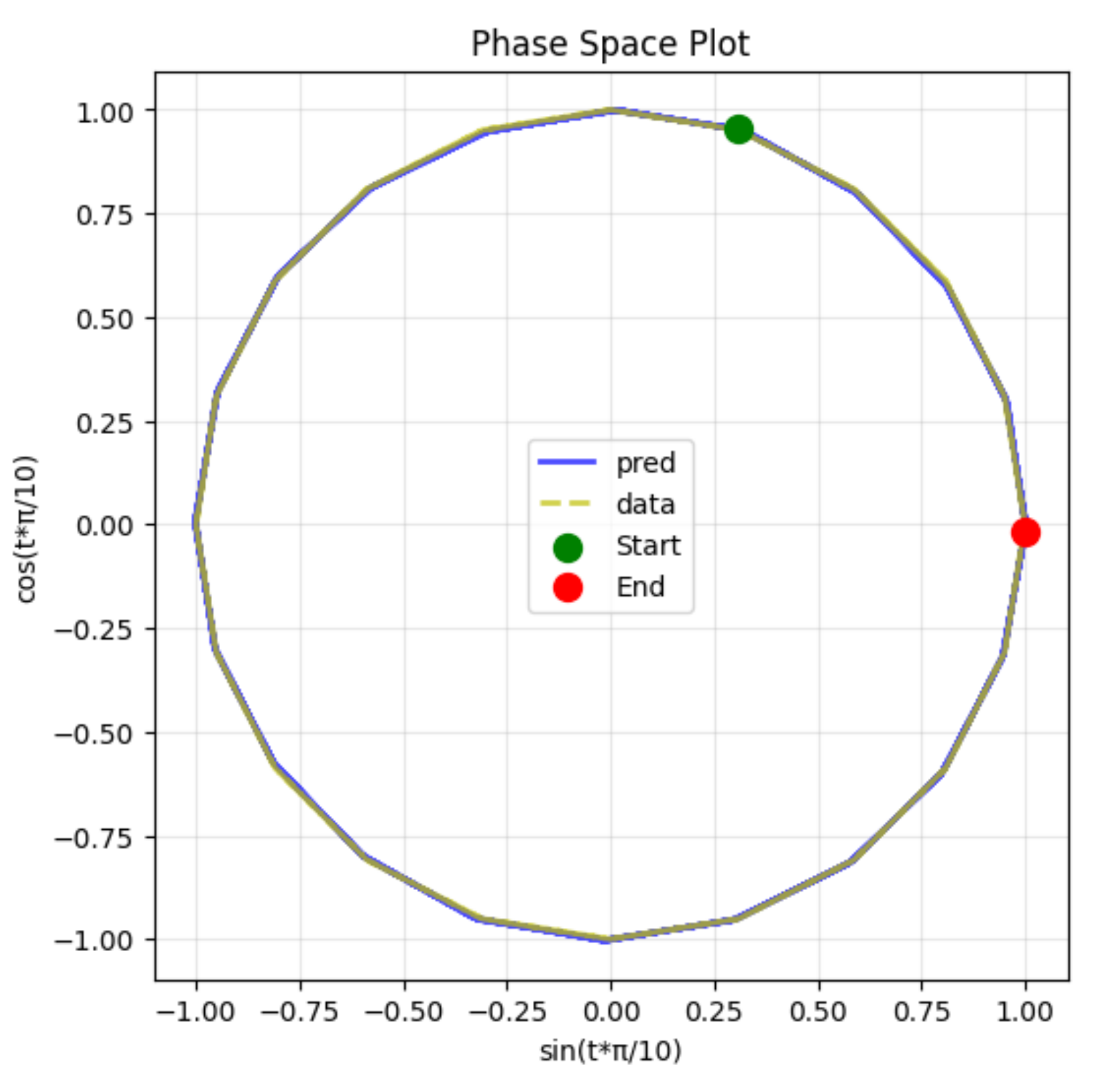
In [41]:

```python
#%% VISUALIZATION OF RESULTS
predictions = generate_traj(model,prediction_length)
print(f"Data shape: {predictions.shape}")
print(f"Time steps: {predictions.shape[0]}")

# Plot the input data
plt.figure(figsize=(10, 4))
plt.plot(predictions[:, 0],'--', label='dim 1 pred', linewidth=2)
plt.plot(predictions[:, 1],'--', label='dim 2 pred', linewidth=2)
plt.plot(data[:, 0], label='sin(t*π/10)', linewidth=2)
plt.plot(data[:, 1], label='cos(t*π/10)', linewidth=2)
plt.xlabel('Time step')
plt.ylabel('Value')
plt.title('predictions')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Plot phase space (sin vs cos)
plt.figure(figsize=(6, 6))
plt.plot(predictions[:, 0], predictions[:, 1], 'b-', linewidth=2, alpha=0.7,label='pred')
plt.plot(data[:, 0], data[:, 1], '--',c='y', linewidth=2, alpha=0.7,label='data')
plt.scatter(predictions[0, 0], predictions[0, 1], c='green', s=100, label='Start', zorder=5)
plt.scatter(predictions[-1, 0], predictions[-1, 1], c='red', s=100, label='End', zorder=5)
plt.xlabel('sin(t*π/10)')
plt.ylabel('cos(t*π/10)')
plt.title('Phase Space Plot')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()
```

```
Data shape: torch.Size([205, 2])
Time steps: 205
```


predictions





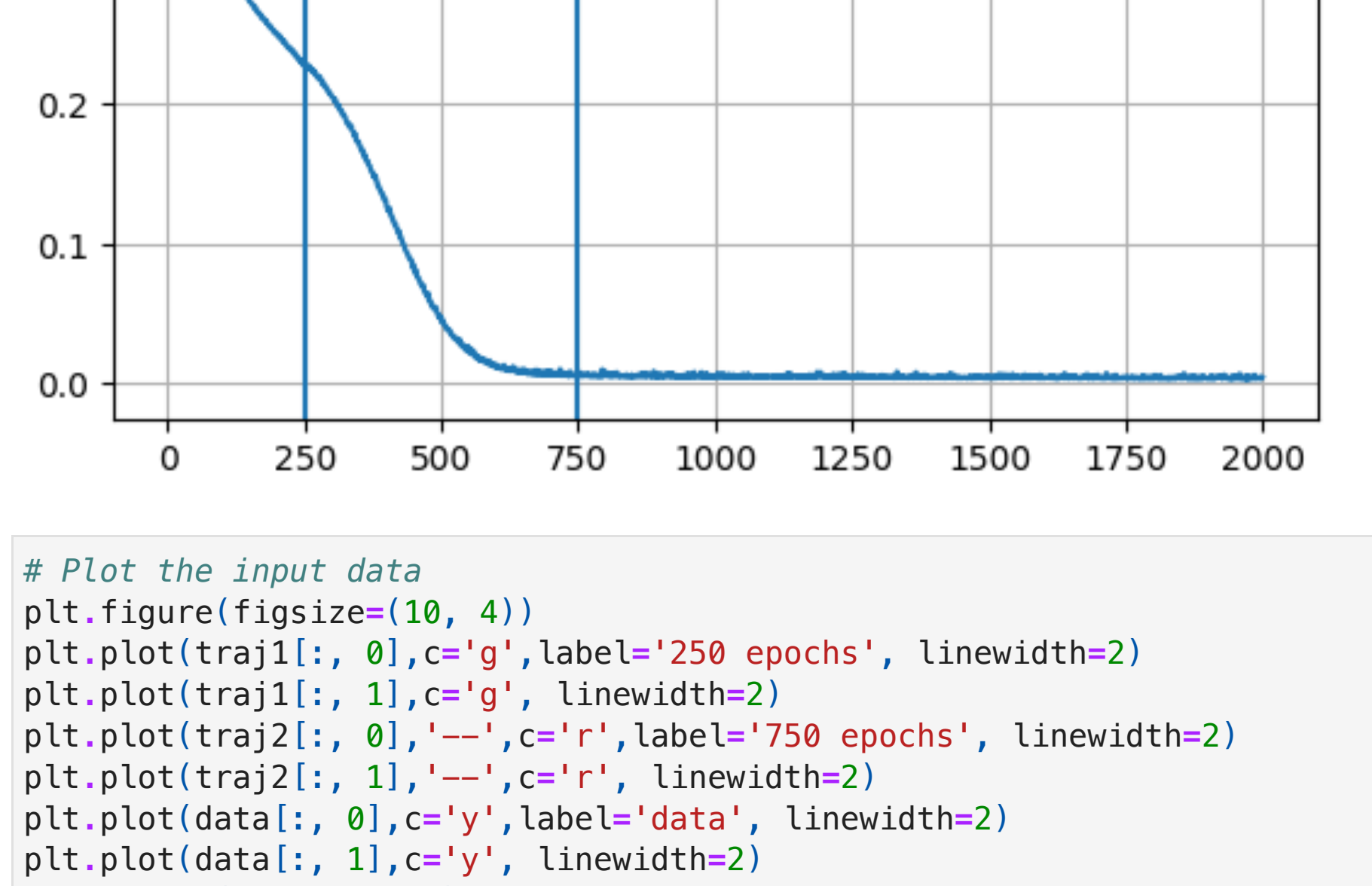
Smallest model with perfect reconstruction for us was hidden size 4, with hidden size 2 we got oscillation but not perfect circular in phase space meaning the amplitudes were a bit off.

Investigating possible bifurcation, from 3.4

In [38]:

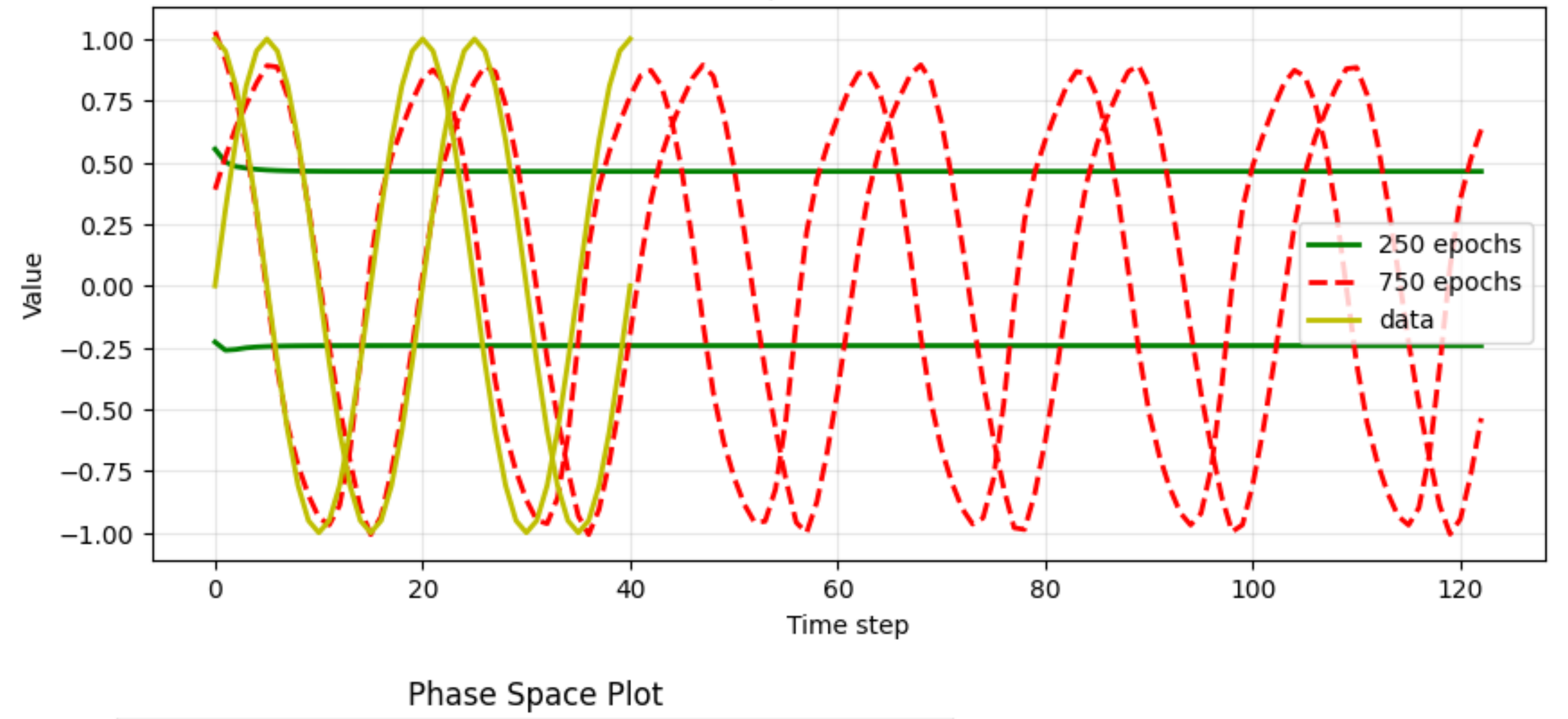
```
def get_idx(row, col, ncols):
    return row * ncols + col

# Example usage:
ncols = 3
row = 2-1
col = 3-1
idx = get_idx(row, col, ncols)
print(idx)
params = hyperparam_list[idx]
print(params)

model = LatentRNN(observation_size, hidden_size)

losses250 = train(model,epochs=250,hidden_size=hidden_size,**params)

traj1 = generate_traj(model,3 * data.shape[0])

losses500 = train(model,epochs=500,hidden_size=hidden_size,**params)

traj2 = generate_traj(model,3 * data.shape[0])

losses1250 = train(model,epochs=1250,hidden_size=hidden_size,**params)

plt.plot(losses250+losses500+losses1250,label='loss curve')
plt.axvline(x=250,label='generated traj 1')
plt.axvline(x=750,label='generated traj 2')
plt.legend()
plt.grid(True)
plt.show()
```

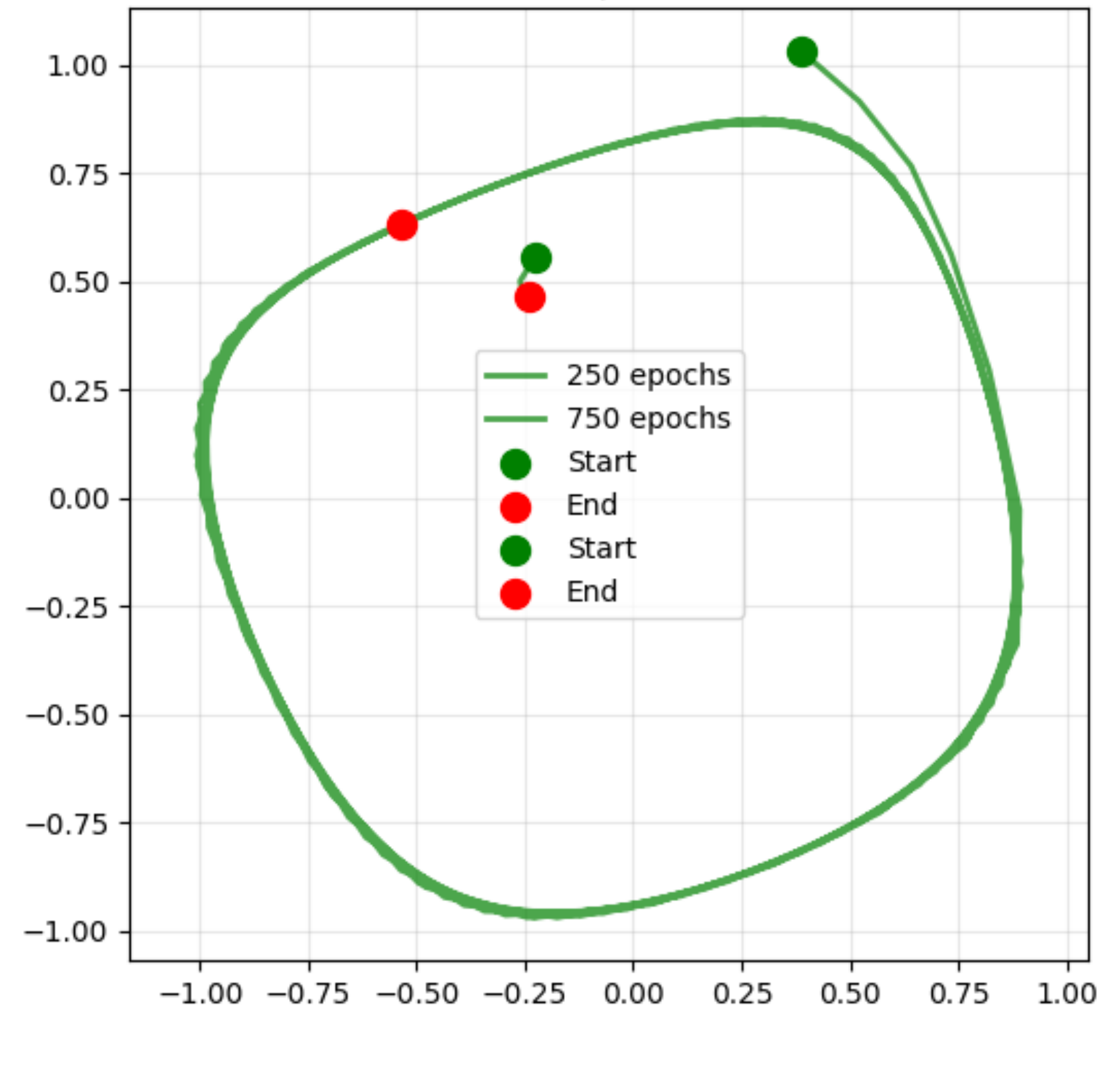
```
5
{'learning_rate': 0.001, 'moment': 0.9, 'optimizer_function': 'SGD', 'batch_size': 5, 'batch_sequence_length': 15, 'minibatc
h': False}

Starting training for 250 epochs...
Optimizer: SGD, LR: 0.001, Batch size: 5, Sequence length: 15
Epoch: 0 loss 0.588704
Epoch: 10 loss 0.562075
Epoch: 20 loss 0.518118
Epoch: 30 loss 0.474538
Epoch: 40 loss 0.436134
Epoch: 50 loss 0.405241
Epoch: 60 loss 0.378851
Epoch: 70 loss 0.358918
Epoch: 80 loss 0.340719
Epoch: 90 loss 0.326134
Epoch: 100 loss 0.314138
Epoch: 110 loss 0.303727
Epoch: 120 loss 0.294937
Epoch: 130 loss 0.287181
Epoch: 140 loss 0.280962
Epoch: 150 loss 0.275141
Epoch: 160 loss 0.269209
Epoch: 170 loss 0.263690
Epoch: 180 loss 0.260170
Epoch: 190 loss 0.254861
Epoch: 200 loss 0.249772
Epoch: 210 loss 0.245670
Epoch: 220 loss 0.241021
Epoch: 230 loss 0.236678
Epoch: 240 loss 0.232166

Starting training for 500 epochs...
Optimizer: SGD, LR: 0.001, Batch size: 5, Sequence length: 15
Epoch: 0 loss 0.227987
Epoch: 10 loss 0.225347
Epoch: 20 loss 0.221999
Epoch: 30 loss 0.217007
Epoch: 40 loss 0.212445
Epoch: 50 loss 0.205586
Epoch: 60 loss 0.199424
Epoch: 70 loss 0.192258
Epoch: 80 loss 0.186681
Epoch: 90 loss 0.179464
Epoch: 100 loss 0.171693
Epoch: 110 loss 0.163718
Epoch: 120 loss 0.155603
Epoch: 130 loss 0.146322
Epoch: 140 loss 0.136400
Epoch: 150 loss 0.127018
Epoch: 160 loss 0.119394
Epoch: 170 loss 0.109257
Epoch: 180 loss 0.099616
Epoch: 190 loss 0.091504
Epoch: 200 loss 0.081213
Epoch: 210 loss 0.073334
Epoch: 220 loss 0.064850
Epoch: 230 loss 0.058961
Epoch: 240 loss 0.054199
Epoch: 250 loss 0.044971
Epoch: 260 loss 0.039088
Epoch: 270 loss 0.034335
Epoch: 280 loss 0.029977
Epoch: 290 loss 0.025713
Epoch: 300 loss 0.023082
Epoch: 310 loss 0.019941
Epoch: 320 loss 0.016694
Epoch: 330 loss 0.014822
Epoch: 340 loss 0.013235
Epoch: 350 loss 0.012262
Epoch: 360 loss 0.010931
Epoch: 370 loss 0.009704
Epoch: 380 loss 0.008558
Epoch: 390 loss 0.008989
Epoch: 400 loss 0.008131
Epoch: 410 loss 0.006282
Epoch: 420 loss 0.006978
Epoch: 430 loss 0.008774
Epoch: 440 loss 0.005829
Epoch: 450 loss 0.008170
Epoch: 460 loss 0.005351
Epoch: 470 loss 0.005519
Epoch: 480 loss 0.006951
Epoch: 490 loss 0.005441

Starting training for 1250 epochs...
Optimizer: SGD, LR: 0.001, Batch size: 5, Sequence length: 15
Epoch: 0 loss 0.004736
Epoch: 10 loss 0.005065
Epoch: 20 loss 0.005444
Epoch: 30 loss 0.004897
Epoch: 40 loss 0.005319
Epoch: 50 loss 0.007869
Epoch: 60 loss 0.006799
Epoch: 70 loss 0.007501
Epoch: 80 loss 0.004967
Epoch: 90 loss 0.004499
Epoch: 100 loss 0.004953
Epoch: 110 loss 0.006138
Epoch: 120 loss 0.006701
Epoch: 130 loss 0.005219
Epoch: 140 loss 0.005149
Epoch: 150 loss 0.005076
Epoch: 160 loss 0.004224
Epoch: 170 loss 0.004707
Epoch: 180 loss 0.004217
Epoch: 190 loss 0.004272
Epoch: 200 loss 0.004277
Epoch: 210 loss 0.005174
Epoch: 220 loss 0.004738
Epoch: 230 loss 0.004259
Epoch: 240 loss 0.004833
Epoch: 250 loss 0.003959
Epoch: 260 loss 0.004157
Epoch: 270 loss 0.004135
Epoch: 280 loss 0.004289
Epoch: 290 loss 0.004661
Epoch: 300 loss 0.004180
Epoch: 310 loss 0.006693
Epoch: 320 loss 0.004357
Epoch: 330 loss 0.004546
Epoch: 340 loss 0.004469
Epoch: 350 loss 0.004438
Epoch: 360 loss 0.004302
Epoch: 370 loss 0.004542
Epoch: 380 loss 0.003940
Epoch: 390 loss 0.005674
Epoch: 400 loss 0.005325
Epoch: 410 loss 0.004025
Epoch: 420 loss 0.004426
Epoch: 430 loss 0.004342
Epoch: 440 loss 0.005601
Epoch: 450 loss 0.004289
Epoch: 460 loss 0.004444
Epoch: 470 loss 0.004612
Epoch: 480 loss 0.004416
Epoch: 490 loss 0.003668
Epoch: 500 loss 0.004341
Epoch: 510 loss 0.004078
Epoch: 520 loss 0.006088
Epoch: 530 loss 0.004738
Epoch: 540 loss 0.005133
Epoch: 550 loss 0.004128
Epoch: 560 loss 0.004807
Epoch: 570 loss 0.003824
Epoch: 580 loss 0.005476
Epoch: 590 loss 0.004434
Epoch: 600 loss 0.003557
Epoch: 610 loss 0.004932
Epoch: 620 loss 0.005142
Epoch: 630 loss 0.004266
Epoch: 640 loss 0.004041
Epoch: 650 loss 0.004055
Epoch: 660 loss 0.003458
Epoch: 670 loss 0.004470
Epoch: 680 loss 0.004374
Epoch: 690 loss 0.003783
Epoch: 700 loss 0.003327
Epoch: 710 loss 0.003190
Epoch: 720 loss 0.004321
Epoch: 730 loss 0.004644
Epoch: 740 loss 0.003418
Epoch: 750 loss 0.003615
Epoch: 760 loss 0.004860
Epoch: 770 loss 0.005701
Epoch: 780 loss 0.004907
Epoch: 790 loss 0.003604
Epoch: 800 loss 0.004914
Epoch: 810 loss 0.003438
Epoch: 820 loss 0.004231
Epoch: 830 loss 0.003961
Epoch: 840 loss 0.003197
Epoch: 850 loss 0.003919
Epoch: 860 loss 0.003367
Epoch: 870 loss 0.003221
Epoch: 880 loss 0.004395
Epoch: 890 loss 0.003609
Epoch: 900 loss 0.003839
Epoch: 910 loss 0.004636
Epoch: 920 loss 0.003482
Epoch: 930 loss 0.003721
Epoch: 940 loss 0.003013
Epoch: 950 loss 0.003161
Epoch: 960 loss 0.003279
Epoch: 970 loss 0.003443
Epoch: 980 loss 0.003083
Epoch: 990 loss 0.003088
Epoch: 1000 loss 0.003080
Epoch: 1010 loss 0.004674
Epoch: 1020 loss 0.004243
Epoch: 1030 loss 0.005440
Epoch: 1040 loss 0.003792
Epoch: 1050 loss 0.003073
Epoch: 1060 loss 0.003060
Epoch: 1070 loss 0.002953
Epoch: 1080 loss 0.004189
Epoch: 1090 loss 0.003165
Epoch: 1100 loss 0.003116
Epoch: 1110 loss 0.003276
Epoch: 1120 loss 0.003249
Epoch: 1130 loss 0.004185
Epoch: 1140 loss 0.003075
Epoch: 1150 loss 0.003614
Epoch: 1160 loss 0.003503
Epoch: 1170 loss 0.004646
Epoch: 1180 loss 0.004157
Epoch: 1190 loss 0.003282
Epoch: 1200 loss 0.005118
Epoch: 1210 loss 0.002936
Epoch: 1220 loss 0.002799
Epoch: 1230 loss 0.003075
Epoch: 1240 loss 0.003475
```



In [39]:

```
# Plot the input data
plt.figure(figsize=(10, 4))
plt.plot(traj1[:, 0],c='g',label='250 epochs', linewidth=2)
plt.plot(traj1[:, 1],c='g', linewidth=2)
plt.plot(traj2[:, 0],'--',c='r',label='750 epochs', linewidth=2)
plt.plot(traj2[:, 1],'--',c='r', linewidth=2)
plt.plot(data[:, 0],c='y',label='data', linewidth=2)
plt.plot(data[:, 1],c='y', linewidth=2)
plt.xlabel('Time step')
plt.ylabel('Value')
plt.title('predictions')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Plot phase space (sin vs cos)
plt.figure(figsize=(6, 6))
plt.plot(traj1[:, 0], traj1[:, 1], c='g', linewidth=2, alpha=0.7,label='250 epochs')
plt.plot(traj2[:, 0], traj2[:, 1],'',c='g', linewidth=2, alpha=0.7,label='750 epochs')
plt.scatter(traj1[0, 0], traj1[0, 1], c='green', s=100, label='Start', zorder=5)
plt.scatter(traj1[-1, 0], traj1[-1, 1], c='red', s=100, label='End', zorder=5)
plt.scatter(traj2[0, 0], traj2[0, 1], c='green', s=100, label='Start', zorder=5)
plt.scatter(traj2[-1, 0], traj2[-1, 1], c='red', s=100, label='End', zorder=5)
plt.title('Phase Space Plot')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()
```





Before the bifurcation in the loss curve the periodic motion of the sines is not yet learned. After the sudden dropp in the loss, which we suspect to be after a bifurcation, the periodic motion is learned.